

Impacto de las Épocas Offline en la Convergencia y Rendimiento de Entrenamientos Distribuidos con O.N.O.

Marcos Bianchi

Facultad de Ingeniería, Universidad de Buenos Aires
Aprendizaje Profundo
Julio 2026

Resumen—Retrasar la sincronización entre nodos en un entrenamiento distribuido reduce el impacto de la red pero induce divergencia entre los pesos (*client drift*). Este trabajo evalúa ese impacto de variar las épocas offline (E) utilizando el sistema de entrenamiento O.N.O. empleando *Parameter Server* sincrónico y *All-Reduce* (*ring*). Entrenando LeNet-5 sobre MNIST FULL en un cluster simulado mediante Docker en una sola computadora, los resultados muestran que incrementar E no reduce el tiempo total debido a la contención del CPU, mientras que degrada la *accuracy* final e incrementa la inestabilidad del entrenamiento.

Palabras clave—Épocas Offline, *Parameter Server*, *All-Reduce*, O.N.O., *Client Drift*.

I. INTRODUCCIÓN Y OBJETIVOS

Permitir que los nodos operen de forma autónoma durante algunas épocas posterga el intercambio de parámetros, disminuyendo la dependencia de la red y priorizando el cómputo local. Sin embargo, demorar la sincronización introduce divergencia en los pesos de los nodos, un fenómeno conocido como *client drift* [1].

Pregunta de investigación. ¿Cómo impactan las épocas offline a la velocidad y eficacia del modelo entrenado?

Objetivos. Evaluar esta pregunta utilizando O.N.O., un sistema de entrenamiento distribuido de modelos de aprendizaje profundo implementado como parte de un trabajo de fin de carrera. Con ello ejecutar varios entrenamientos sobre una misma arquitectura, dataset e hiperparámetros para relevar las diferencias en cuanto al impacto real de las épocas offline.

II. ARQUITECTURA DEL SISTEMA Y MECANISMOS DE SINCRONIZACIÓN

II-A. Componentes Principales de O.N.O.

- **Orchestrator:** Mediante una configuración declarativa del entrenamiento brindada por el usuario, configura al resto de nodos para iniciar el entrenamiento de un modelo usando alguno de los algoritmos implementados. Al finalizar un entrenamiento, consolida los resultados parciales generando un archivo *safetensors* con el resultado final que contiene los parámetros del modelo entrenado. Para una mejor automatización, cuenta con una interfaz FFI de Python que fue utilizada para simular los ensayos discutidos más adelante.

- **Workers:** Computan gradientes locales por backpropagation sobre una partición asignada del dataset. En *All-Reduce* aplican localmente el optimizador mientras que en *Parameter server* delegan la actualización a los servidores.
- **Servers:** Nodos dedicados al guardado y optimización de parámetros del modelo entrenado. Reciben gradientes parciales de los workers, los agregan y aplican sobre los parámetros para luego redistribuirlos y volver a empezar.

II-B. Dinámica de Sincronización Demorada ($E > 1$)

Al parametrizar $E > 0$, los nodos entrenan aislados durante múltiples ciclos. El tráfico de red se reduce considerablemente de forma inversamente proporcional a E . No obstante, la falta de sincronización de estado causa que los pesos locales de cada uno de los workers diverjan (*client drift*). Al alcanzar la etapa de sincronización, la consolidación se realiza por el promedio directo de los gradientes parciales.

Bajo un entrenamiento optimizado por *Descenso por Gradiente*, una ecuación que describe la etapa de optimización es:

$$W_{t+1} = W_t - \frac{\lambda}{N} \sum_{i=1}^N G_i \quad (1)$$

II-C. Algoritmos de Entrenamiento Distribuido

El sistema implementa varios algoritmos de entrenamiento distribuido, en particular el informe se va a centrar en los siguientes:

- **Parameter Server Sincrónico:** Un algoritmo centralizado bajo el modelo propuesto por Li et al. [2] donde una barrera bloquea a los *workers* hasta que cada *server* procesa el lote global de gradientes, garantizando que todos los *workers* estén en todo momento calculando gradientes de la misma epoch.
- **All-Reduce:** Un algoritmo descentralizado inspirado en arquitecturas de anillo de alto rendimiento como Horovod [3], donde los *workers* se organizan en una topología de anillo lógico compartiendo fragmentos de gradientes en fases sucesivas de scatter y gather, calculando de forma idéntica y local el vector de parámetros resultante.

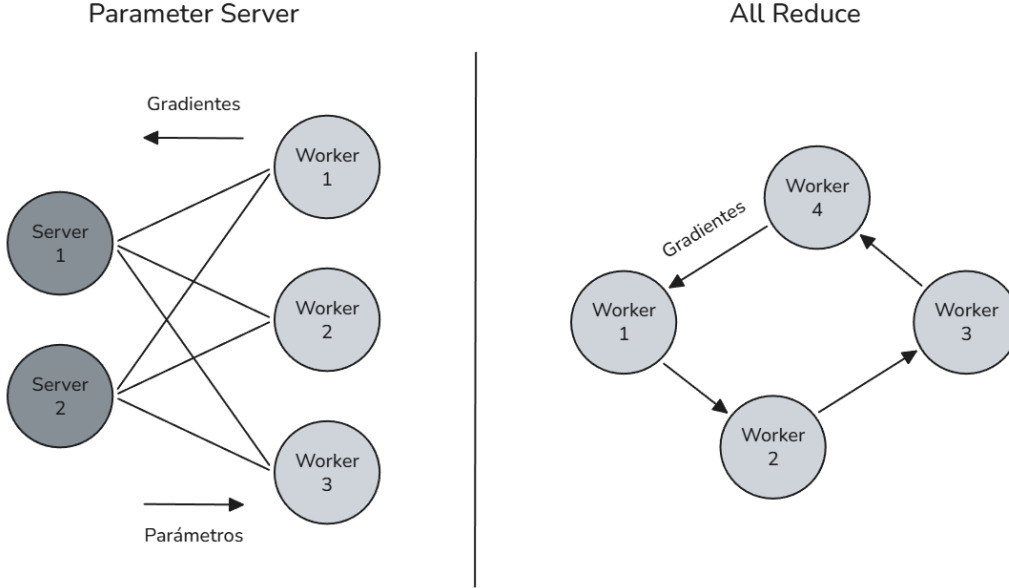


Figura 1: Topologías distribuidas soportadas en O.N.O. (izquierda) Arquitectura centralizada bajo *Parameter server* donde los *workers* obtienen parámetros y envían gradientes a *servers* fragmentados. (derecha) Topología descentralizada en anillo *All-Reduce* donde cada nodo actúan como *worker* comunicándose con sus nodos adyacentes.

III. METODOLOGÍA EXPERIMENTAL

Todos los entrenamientos fueron realizados en la misma computadora de forma secuencial utilizando Docker para simular un entorno multicomputadora, sumando un total de 10,50 horas de ejecución. Las especificaciones técnicas del entorno se detallan en la Tabla II, mientras que las variables lógicas y configuraciones del modelo O.N.O. se consolidan en la Tabla I.

Tabla I: Configuración de Ensayos

Categoría	Parámetro	Valores Evaluados
Modelo y Datos	Dataset	MNIST (Full) [4]
	Arquitectura de Red	LeNet-5
	Función de Pérdida	Entropía Cruzada
Optimización	Optimizador	Adam
		$\alpha = 0,01$
		$\beta_1 = 0,9$
		$\beta_2 = 0,999$
	Tamaño de Mini-Batch	64
	Épocas Máximas	48
	$\epsilon = 10^{-8}$	
Infraestructura	Serialización	Sparse ($r = 0,5$)
Variables Libres	Algoritmo Distribuido	All-Reduce, Parameter Server (Síncrono/Bloqueante)
	Nodos	2, 4, 6, 8, 10
	Épocas Offline	0, 1, 2, 3, 4

optimizar la comunicación distribuida limitando el intercambio de sectores poco impactantes de los gradientes [5]. El parámetro r sirve para acotar el sampleo del gradiente que se quiere transmitir: a mayor r , mayor será la compresión teórica.

El algoritmo calcula la cardinalidad efectiva del mensaje mediante:

$$k = \text{round}(|g| \cdot (1,0 - r)) \quad (2)$$

El valor resultante k se utiliza para escoger los k elementos de mayor magnitud absoluta dentro del tensor de gradientes local, fijando un umbral mínimo. Todo valor que alcance o supere dicho umbral se incorporará al mensaje por enviar. De esta forma, un valor de $r = 0,0$ samplea la totalidad del gradiente, mientras que un $r = 0,5$ (como el configurado para estos experimentos) filtra el 50,00 % de los componentes para disminuir el tránsito por red. Los elementos descartados se acumulan en un gradiente residual para ser considerados en el próximo mensaje.

III-A. Compresión de Red: Serialización Sparse

La configuración de la sección de infraestructura responde al método de serialización de gradientes del sistema, el cual soporta una variante *sparse* (o dispersa) diseñada para

Tabla II: Entorno de Ejecución

Categoría	Componente	Detalle
Procesador	Modelo	Intel Core i5-8350U
	Arquitectura	x86_64
	Núcleos Físicos	4
	Hilos Lógicos	8
	Frecuencia de Reloj	Base: 1,70 GHz Turbo: 3,60 GHz
Memoria	Capacidad Total RAM	8 GB
	Tipo	DDR4
	Velocidad	2400,00 MHz
Entorno de Software	Sistema Operativo	Ubuntu 24.04 LTS
	Entorno de Ejecución	Docker v29.6.1
		Rust v1.96.1
		CPython v3.14.0

IV. RESULTADOS Y ANÁLISIS EXPERIMENTAL

IV-A. Análisis Temporal y Escalabilidad del Hardware

La variación de E no trajo ningún tipo de mejora apreciable en el tiempo de ejecución de los entrenamientos, las curvas de cada configuración se solapan en todas las pruebas. Dado que el sistema se ejecuta sobre una sola máquina tiene sentido que no haya una gran diferencia en tiempo, aunque sí me esperaba algún tipo de mejora.

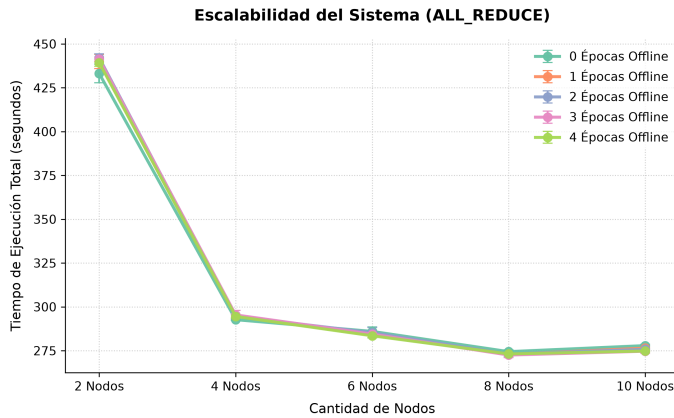


Figura 2: Comparación de tiempos de ejecución bajo distintas cantidades de Épocas Offline con All-Reduce.

Como se observa en la Figura 2, el rendimiento escalable se estanca al superar los 4 nodos. En *All-Reduce*, escalar a 10 nodos llegó a degradar la performance debido al alto nivel de contención sobre el CPU.

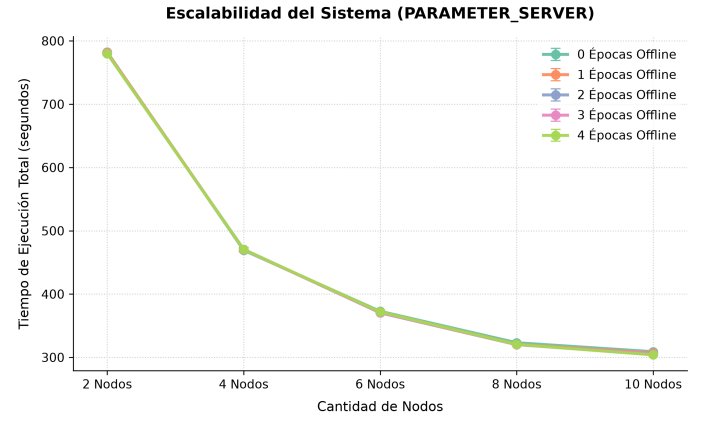


Figura 3: Comparación de tiempos de ejecución bajo distintas cantidades de Épocas Offline con Parameter Server.

Se aprecia facilmente en la Figura 3 que el sistema aún con 10 nodos (5 *workers* y 5 *servers*) consigue mejorar su tiempo previo con menor cantidad de nodos. A diferencia de *All-Reduce*, donde los nodos están constantemente procesando datos, *Parameter Server* tiene momentos de tiempo muerto, por ejemplo, en la variación sincrónica utilizada, donde en el momento en el que el *server* está optimizando el modelo sabemos que el resto de *workers* tienen que estar bloqueados en la barrera interna del servidor. Esto le cede hilos de ejecución a los nodos *server*, por lo que se aprovecha mejor tener más de un tipo de entidad por computadora, o en este caso, CPU.

Es importante notar que ya de entrada *All-Reduce* es más rápido que *Parameter Server*, esto tiene sentido porque *All-Reduce* aprovecha mejor la distribución de responsabilidades y cada nodo está constantemente procesando datos, ya sea calculando gradientes, compartiendolos u optimizando los parámetros.

IV-B. Métricas de Exactitud

En cuanto a resultados de accuracy, la variación de E se puede apreciar mucho mejor ya que no depende del hardware en donde se esté ejecutando el entrenamiento. Se aprecian mucho mejor estos resultados que los anteriores.

Consistentemente para ambos algoritmos el modelo resultante es peor en cuanto a accuracy a medida que E incrementa, lo cual no me sorprende para nada, en cuanto a qué tan peor es, teniendo en cuenta que introducir tan solo 1 época offline ya reduce la cantidad de sincronizaciones en la red a la mitad, este deterioro es tan solo del 1 %.

Si los entrenamientos fueran ejecutados en un entorno real, se podría comprender mejor que tan superior es incrementar E , por ejemplo, llegar a ejecutar más épocas totales en la misma cantidad de tiempo, ¿Podría un setup como ese superar a uno que sincronize en todas las épocas?

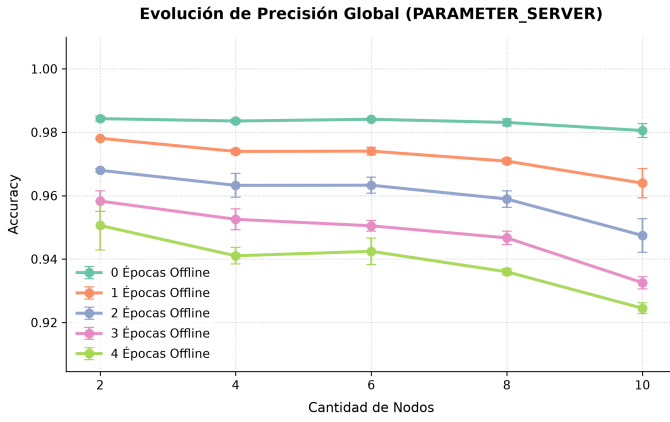


Figura 4: Comparación de accuracy bajo distintas cantidades de Épocas Offline con Parameter Server.

Como mencioné antes, se nota un degradamiento en la accuracy a mayor E . Lo que si puede ser interesante es que a medida que incrementa la cantidad de nodos, esto se hace más notable, lo cual tiene sentido si pensamos que a medida que la cantidad de nodos aumenta, también aumentan las particiones de datasets y esto las hace cada vez más chicas. El *Orchestrator* termina repartiendo particiones de dataset cada vez más chicas y potencialmente más sesgadas, por lo que tener menos sincronización de parámetros en los *workers* termina perjudicando aún más al fin del día.

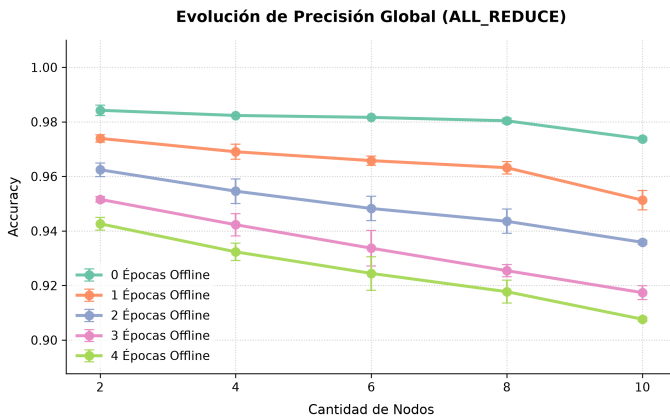


Figura 5: Comparación de accuracy bajo distintas cantidades de Épocas Offline con All-Reduce.

Hablando de *All-Reduce*, la degradación de la accuracy en relación a la cantidad de nodos y E es todavía más notable que en *Parameter Server*. En cada uno de los setups empleados, *All-Reduce* siempre tuvo el doble de *workers* (ya que no tiene *servers*). Esto implica dividir más los datasets y aún más independencia de cada uno de ellos con respecto al resto.

Sincronizar los parámetros y/o gradientes de forma tan tardía también provoca que sus promediados introduzcan correcciones más fuertes y abruptas, generando una desestabilización en el descenso por el gradiente de la función de pérdida y resultando en un peor modelo resultante.

IV-C. Inestabilidad de Pérdida durante el Entrenamiento

Mediante las historias de pérdida de los entrenamientos realizados, se seleccionan algunos casos representativos para relevar algunos puntos importantes. Como mencioné anteriormente, esperaba que incrementar E introdujera inestabilidad en el entrenamiento.

Para los entrenamientos realizados en *All-Reduce* y *Parameter Server* de 2 nodos se nota esta inestabilidad en forma de picos cada vez más pronunciados a medida que crece E .

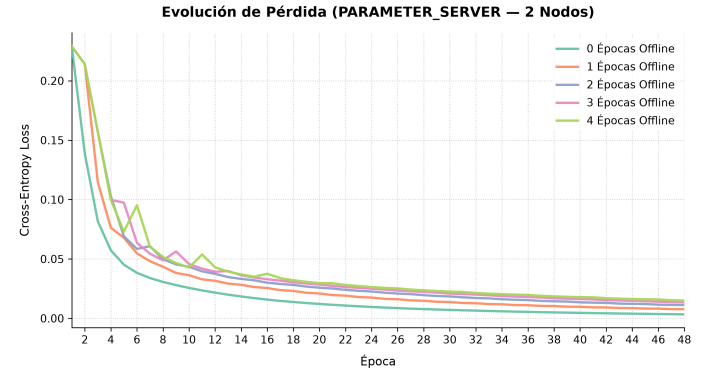


Figura 6: Trayectoria de Loss en Entropía Cruzada exhibiendo ruido estocástico por cómputo offline en Parameter Server (2 nodos).

Es claro que al optimizador le cuesta más al comienzo, ya que suele ser la etapa de corrección más grande del entrenamiento, una vez que encuentra un mínimo local se estabiliza y converge. Notar que cada vez converge a valores más altos, ¿Será que a mayor E no es posible o dificulta al optimizador de alguna forma a no alcanzar nunca el mínimo local verdadero? Al estilo de tener un alto valor de *learning rate* en el que el optimizador podría quedar oscilando sobre el mínimo sin nunca alcanzarlo.

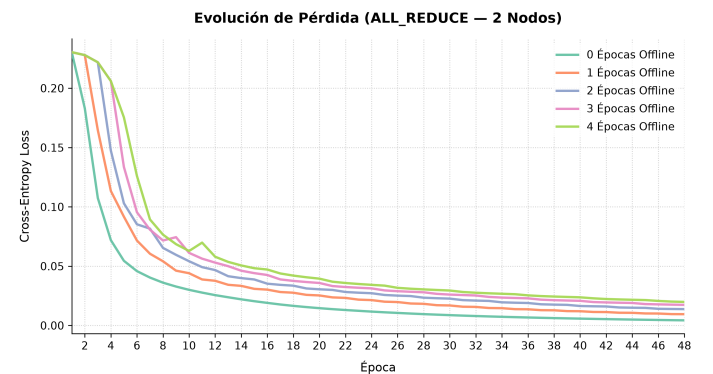


Figura 7: Trayectoria de Loss en Entropía Cruzada exhibiendo ruido estocástico por cómputo offline en All-Reduce (2 nodos).

Para *All-Reduce* sucede algo parecido aunque parece más controlado, sinceramente esperaba lo contrario, hallar mayor inestabilidad con este algoritmo que con *Parameter Server*, se ve que se mantuvo más estable.

Siendo que la desestabilidad perjudica a todo el entrenamiento, capaz es interesante preguntarse en qué momento conviene aplicar las épocas offline, a priori podría suponerse que conviene introducirlas cuando el entrenamiento está más estable, cuando el grueso de la optimización ya pasó, con la esperanza de que algún *worker* haya encontrado un muy buen mínimo local para desestabilizar al resto y llevarselos con él.

V. DISCUSIÓN Y LIMITACIONES

Las ventaja de introducir épocas offline es estrictamente bajar el tiempo del entrenamiento ejecutado. En un entorno local simulado, eliminar ese overhead de comunicación no compensa al costo que resulta de un peor modelo. Por lo menos en estos experimentos no se evidenciaron ventajas de utilizar un E mayor a 0.

En contraposición, en un setup multi-computadora sobre una red física, retrasar la agregación de gradientes y las sincronizaciones de nodos puede resultar crítico para amortizar el tiempo de comunicación. Al final del día se tendrá que tomar en cuenta el tamaño del modelo, la velocidad de la red y cuantos nodos se están utilizando.

Es importante notar que con modelos más grandes, los mensajes (que contienen gradientes y parámetros) van a ser grandes también. Para entrenamientos de mucho dataset y poco modelo, el cuello de botella va a ser el cómputo de gradientes y optimización del modelo. En cambio, entrenamientos de poco dataset y mucho modelo, donde la comunicación toma más protagonismo, incrementar E puede ser útil para bajar los tiempos de ejecución y realizar más épocas por segundo.

VI. CONCLUSIÓN

El uso de épocas offline en O.N.O. altera el balance entre comunicación y fidelidad algorítmica en el entrenamiento. En la infraestructura simulada, incrementar E perjudicó al modelo entrenado de manera predecible pero sin aportar ganancias de velocidad, debido a limitaciones del hardware empleado.

Como trabajo futuro, se plantea validar el sistema bajo configuraciones multi-computadora reales, comparando distintas arquitecturas de modelos de distintos tamaños variando E .

REFERENCIAS

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial Intelligence and Statistics*. PMLR, 2017, pp. 1273–1282.
- [2] M. Li, L. Zhou, Z. Yang, A. Li, F. Xia, D. G. Andersen, and A. Smola, "Parameter server for distributed machine learning," in *NIPS Workshop on Big Learning*, Lake Tahoe, NV, USA, 2013. [Online]. Available: <https://www.istc-cc.cmu.edu/publications/papers/2013/ps.pdf>
- [3] A. Sergeev and M. Del Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," *arXiv preprint arXiv:1802.05799*, 2018.
- [4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [5] A. F. Aji and K. Heafield, "Sparse communication for distributed gradient descent," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Copenhagen, Denmark: Association for Computational Linguistics, September 2017, pp. 440–445. [Online]. Available: <https://aclanthology.org/D17-1045>